

How Lucida Works

A viva reference. Everything below is the system as it actually stands today — including what is live, what is simulated, and the problems we hit and fixed. If a question is asked about any part of the app, the answer is somewhere in here.

Repository: github.com/EsmeAbha/Business_Marketing_Analysis_LLM 47 Python modules · 16,700 lines · 24 database tables · 33 automated tests · 50 commits

1. What it is, in one paragraph

Lucida is an **AI workforce for a small shop**. The owner types in plain language — or drops in a photo of what they make — and a **Supervisor agent** decides which of **eight specialists** should handle it, in what order. They share one memory of the business, and the run **stops and waits for the owner's approval** before anything irreversible: publishing an advert, spending money, booking a courier. It covers the whole lifecycle: deciding what to sell, validating a product, pricing it, recording stock, marketing it, listening to customers, arranging delivery, and reporting back with a revised plan.

It is not a chatbot with tools bolted on. It is a **state machine** (LangGraph) whose nodes are agents, with durable memory and a real suspend/resume mechanism.

2. Running it, for the demo

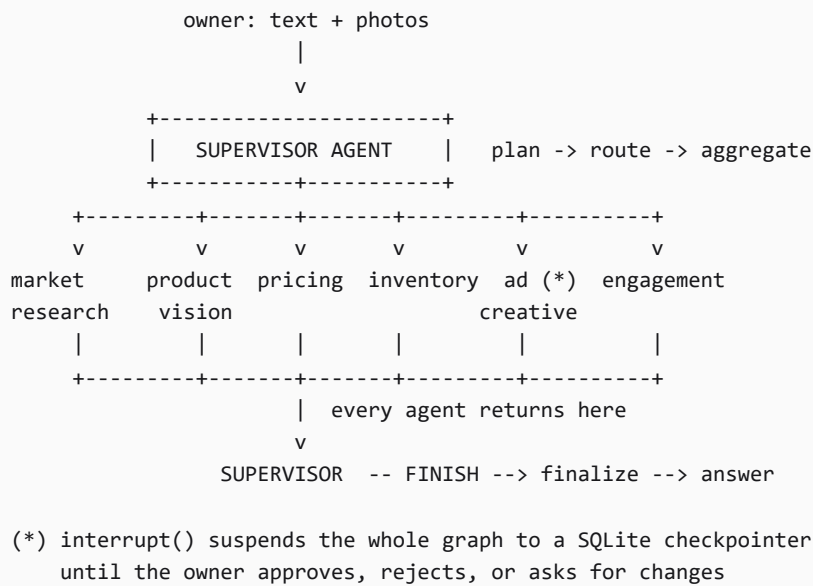
```
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
copy .env.example .env          then set GOOGLE_API_KEY
python serve.py
```

Open `http://127.0.0.1:8000`. Sign up with **a name and a password** — there is no email verification. Tests:

```
python tests/test_workforce.py
```

 (33 tests, needs no API key).

3. Architecture



Star topology, not a pipeline. Every specialist returns to the Supervisor, which re-decides. This is the design decision most worth defending: if the Customer Engagement agent discovers people asking for a product the shop does not stock, the Supervisor can route *backwards* into market research or pricing. A fixed pipeline could only march forwards. The system is reactive to what it learns.

Guards on the loop. `max_supervisor_steps = 18` and LangGraph's `recursion_limit = 60`. A router that returns an unknown agent name, or fails to parse, falls through to `finalize` rather than spinning.

4. The Supervisor

Three jobs, and it never does specialist work itself.

1. **Plan.** On the first turn it reads the owner's message and any photo, works out which lifecycle stage the business is in, and drafts a short plan naming the agent responsible for each step.
2. **Route.** On every turn it picks exactly one agent to run next — with a stated reason — or `FINISH`.
3. **Aggregate.** When done, it composes every specialist's structured output into one answer written for a shop owner, not a reviewer.

The routing guard (important)

The router is told not to re-run an agent that already succeeded. **Models ignore this.** In testing, the Supervisor re-ran `inventory` eight times and `engagement` seven times until the step limit killed the run — **334 seconds and 78,000 tokens for one question.**

So it is enforced in code, not in the prompt:

- an agent that succeeded is removed from the allowed set
- an agent that failed gets exactly **one** retry
- `product_vision` is unroutable when no image was uploaded

- an invalid pick falls forward to the next unfinished step of the plan, else `FINISH`

The same request now takes **25 seconds and 22,000 tokens**. This is the single most useful thing to be able to explain: *prompts express intent, code enforces invariants*.

5. The eight specialists

#	AGENT	WHAT IT DOES	TOOLS IT CALLS
1	Market Research	Finds low-competition, high-demand niches and a competitor price band	Live web search, RAG
2	Product Vision	Identifies a product from the owner's photo , estimates demand and price fit, gives GO / NO-GO	Gemini vision, web search
3	Pricing & Cost	Cost-plus price, margin, break-even, competitor comparison	Sandboxed Python , RAG
4	Inventory	Records stock (product, quantity, cost, photo), flags low stock	SQLite
5	Ad Creation (gated)	Writes platform-specific ad copy, then publishes	Meta Graph API, YouTube
6	Customer Engagement	Reads DMs and comments, sentiment, pre-orders, unmet demand	Telegram (live), Meta
7	Delivery (gated)	Books courier pickup and delivery	Pathao, Steadfast
8	Reporting	Restock alerts, demand shifts, profit analysis, revised plan	RAG over everything, Python

"Gated" means the agent stops for owner approval before it acts.

Every agent returns a **validated Pydantic object**, not prose. That is what lets one agent consume another's result programmatically.

6. How the agents collaborate

Two channels, and the distinction matters.

Channel 1 — the structured message bus

Each hand-off writes a row to `agent_messages` :

```
supervisor -> pricing      "Set a price for the frozen paratha pack"
pricing     -> supervisor  summary + full PricingResult payload
```

Because payloads are typed, fields flow directly: `market_research.competitor_price_low` and `competitor_price_high` become the Pricing agent's inputs; `pricing.recommended_price` becomes the price in the ad copy. **No agent re-parses another's prose.**

Channel 2 — shared memory

Messages are transient. Memory is not. Two stores behind one facade (`SharedMemory`):

- **Structured (SQLite)** — the system of record: profile, products, inventory, stock movements, pricing history, conversations, pre-orders, campaigns, deliveries, reports, approvals. 24 tables.
- **Semantic (vector store)** — every agent's conclusions written as prose, tagged `{agent, kind, session_id}`, retrieved by meaning.

This is what lets the **Reporting agent, running last, ask "what price did we decide and why"** and get the Pricing agent's reasoning from three stages earlier — across process restarts and separate sessions. The Memory screen has a retrieval box that runs exactly this query, so the mechanism is inspectable rather than claimed.

How the embeddings work

Deterministic hashed bag-of-words into a 1024-dimension L2-normalised vector. Three feature families are hashed into the same space: whole words, word bigrams (phrase signal), and **character 4-grams**.

The character n-grams are there for a specific reason. Our first version matched exact tokens only, so the query "what **price** did we **decide**" scored **zero** against a document saying "**Pricing decision**" — retrieval returned nothing at all. Padded character grams (`^pri` , `pric`) bridge the morphology. Hashing uses **CRC32, not Python's `hash()`** , because Python randomises string hashing per process, which would make stored vectors irreproducible after a restart.

No model download, no embedding API spend, works offline.

7. Human-in-the-loop

Implemented with LangGraph's `interrupt()` plus a **SQLite checkpointer** — not a confirmation dialog in the UI. When the Ad agent reaches the publish step, LangGraph persists the entire graph state and unwinds. **The server can restart and the paused run is still recoverable.**

DECISION	EFFECT
Approve	The agent proceeds — publishes, or books the courier
Reject	Action skipped; the draft is still saved for the owner
Request changes	Feedback is fed back into the prompt and the agent rewrites , then asks again (up to 2 revisions)

Request changes is the one worth demonstrating: it is a genuine correction loop, not a yes/no gate. Verified end to end — suspend, request changes, rewrite, re-ask, approve, publish, continue to completion, in 72 seconds.

A subtlety we had to handle. Resuming replays the node from its start, so any side effect before the interrupt happens again. Recording an approval decision was therefore duplicated on every later resume. Fixed with a unique index making that write idempotent.

8. Tools, and the live/simulated boundary

TOOL	HOW IT WORKS	IF NOT CONFIGURED
Web search	Tavily	DuckDuckGo (free), then labelled simulated
Vision	Gemini, base64, auto-downscale over 5 MB	Agent says it could not see the photo
Code execution	Restricted <code>exec</code> : whitelisted builtins, regex deny-list, 10s timeout, no imports, no IO, no network	required
RAG	In-process vector store	required
Telegram	<code>getUpdates</code> long-poll and <code>sendMessage</code>	—
Meta (FB/IG)	Graph API	labelled simulated publish
Courier	Pathao live, Steadfast	labelled simulated consignment

Why the Pricing agent runs real Python. Margins and break-even are computed in a sandbox and then *recomputed* at whatever price the model chose. The model explains the decision; Python owns the arithmetic. The numbers in a business plan cannot be a hallucination.

When a model dies mid-project. Free tiers are rationed and providers retire models without warning — both happened to us. A call moves down a fallback chain when the model is **capped** (429), **retired** (404), or **returns the right JSON but not as a tool call**, an intermittent failure that used to kill a whole run. Only when the chain is exhausted does the owner see a message, and it says when to come back rather than printing a provider traceback.

On simulated adapters. They return the *same shape* as the live ones, so agent logic never branches on which mode is active — only a `simulated` flag differs, and it is surfaced in the trace, the database and the final report. Adding an API key switches to live with **no code change**.

Nothing simulated is ever written into a shop's customer records. An unconnected channel returns *no* messages rather than plausible ones. This was a deliberate change: we originally seeded a sample inbox, and it poisoned the real record — inventing sentiment, phantom pre-orders, and an "urgent" complaint about a late delivery that never existed, which the Reporting agent then repeated to the owner as fact.

9. Observability and cost

Every agent action, tool call, hand-off, LLM call, approval and failure emits a `TraceEvent` — in-memory for live polling, SQLite for durability. **The UI reads the same store the system writes**, so there is no second telemetry path that could drift.

Error containment. Every agent runs inside a boundary: a failure becomes `ok=False` plus an error string in state, which the Supervisor sees on its next routing decision and can route around. One agent failing never crashes a run.

Token and cost accounting. Structured calls use `include_raw=True` so the underlying message survives parsing and its usage metadata can be read. Cached tokens are subtracted from the input count and re-billed at their own multipliers rather than double-charged. Cost is broken down per agent.

10. Multi-tenancy, accounts and sessions

Each shop's data is **its own SQLite file** under `data/shops/<id>/shop.db` — isolated by file, not by an owner column, so a forgotten `WHERE` clause cannot leak one business into another. Verified: a new account re-synced its own Telegram messages rather than seeing the previous shop's.

Sign-in takes **a name and a password**. No email verification: nothing is ever posted to an address, so requiring one — and then proving ownership of it — was a chore that bought no safety. Passwords are bcrypt, minimum eight characters. The session is a **signed 14-day cookie**, marked `Secure` when `AIW_HTTPS=1`.

11. Problems we hit, and how we fixed them

This section is the honest engineering record. Each was found by testing against real APIs, not by reading code.

1. The approval gates were silently dead. `interrupt()` suspends the graph by *raising* `GraphInterrupt`. Our agent error-boundary caught it as a failure, so every human-in-the-loop checkpoint was quietly disabled — the ad agent reported "failed unexpectedly" instead of pausing. Fixed by re-raising `LangGraph` control-flow exceptions before the generic handler. A catch-all that swallows control flow is a trap worth knowing about.

2. The Supervisor looped until the step limit. See section 4. 334s became 25s.

3. RAG retrieved nothing. See section 6. Exact-token matching could not connect "price" to "Pricing".

4. Rate limits, and where the tokens actually go. On a free tier the cap is per-minute *per model*, and the requested `max_tokens` counts against it **before a single prompt token**. An 8,000-token output budget alone exceeded an 8,000 tokens/minute cap, so the first run died on the Supervisor's opening call. Fixed by sizing each call to its schema — a routing decision needs about 200 tokens, not 2,500 — and putting routing on a different model so it draws from a separate bucket.

5. Fabricated customers corrupting the record. See section 8.

6. A secret nearly went into a public repository. A `.env.backup` made while rotating keys was not covered by `.gitignore`, and `git add -A` would have committed a live API key. `.gitignore` now covers `.env.*` and `*.backup`, and the staged diff is scanned for credential patterns before every commit.

12. What is live, and what is waiting

Live, verified against real APIs:

- Chat with saved conversations; **photo understanding** (Gemini)
- Web research; ad generation and editing; product catalogue

- **Telegram as a real customer channel** — a real person messages the bot, the Engagement agent reads and classifies it, and the owner replies from the browser
- Delivery quoting and booking through Pathao's live merchant API
- The workforce graph, approval gates, trace, cost accounting, admin view

Built and tested, waiting on credentials rather than code:

- **Messenger / Facebook / Instagram** — need a Meta app *plus App Review and Business Verification*. In Development mode a Page can only exchange messages with accounts that hold a role on the app, which is why Telegram was added: it has no review process, so a member of the public can message the shop today.
- **YouTube** — an API key only reads; uploading needs an OAuth client
- **Steadfast** — needs a merchant key pair

Known weak: ad artwork, while it runs on the free keyless image model.

13. Limitations we would name before being asked

- **One instance only.** The run lock and approval gates live in process memory, and each shop is a file on disk. It scales up, not out. Two copies would hand one owner two sessions and let two runs overwrite each other. This is why serverless hosts are the wrong shape and the deployment target is Fly.io with a mounted volume.
 - **No password reset.** Without email, a forgotten password means a lost shop.
 - **Output quality tracks the model.** On a free tier the reasoning is visibly weaker than on a frontier model; the architecture is unchanged, the judgement is not.
 - **A long run takes one to two minutes,** mostly rate-limit backoff and two vision passes.
 - **Meta remains simulated,** and is labelled as such everywhere it appears.
-

14. Questions we expect, with answers

Why a supervisor rather than agents calling each other? One place decides, so routing is inspectable and bounded. Peer-to-peer calls make the control flow implicit and unbounded. The Supervisor also gives one place to enforce the invariants in section 4.

How is this different from one prompt with tools? Separation of concerns and durability. Each agent has its own prompt, schema and tool set, so failures are contained and outputs are typed. A single prompt has none of the memory, none of the suspend and resume, and no per-agent cost accounting.

Where is the planning and reasoning? The Supervisor drafts an explicit plan before any work, then re-decides after every agent using what has been learned — including routing backwards when customer signal contradicts the original plan.

What actually happens when I click Approve? LangGraph resumes from the checkpoint with `Command(resume=...)`. The node re-enters, `interrupt()` returns your decision instead of raising, and the agent proceeds. The decision is written to `approvals` and appears in the trace.

Show me agents sharing memory. Memory screen, retrieval box, type "what price did we decide and why". It returns the Pricing agent's own reasoning, retrieved by meaning, written in an earlier stage.

How do you know the numbers are right? They are computed in a Python sandbox and recomputed at the chosen price, then written to `pricing_history`. Low-stock alerts and sentiment counts are likewise recomputed from the database after the model answers, so what is displayed cannot drift from the record.

What did you personally get wrong and fix? Section 11. The interrupt-swallowing bug is the best answer: it was invisible, it disabled the headline feature, and it was only found by running the flow end to end.

15. Where things are in the repository

PATH	WHAT IS IN IT
<code>serve.py</code>	The web server: routes, sessions, the run lock
<code>src/lucida/graph.py</code>	Graph assembly and the runtime (start, resume, retry)
<code>src/lucida/supervisor.py</code>	Planner, router, routing guard, aggregator
<code>src/lucida/agents/</code>	The eight specialists, <code>base.py</code> , and the Pydantic schemas
<code>src/lucida/memory/</code>	<code>db.py</code> (SQLite), <code>vector.py</code> (embeddings), <code>shared.py</code> (facade)
<code>src/lucida/tools/</code>	Web search, vision, code sandbox, channels, courier, inbox
<code>src/lucida/observability.py</code>	Logging, trace bus, error containment
<code>src/lucida/pricing.py</code>	Token accounting and cost estimation
<code>web/</code>	Auth, screens, admin, and the bridge that feeds the design
<code>tests/test_workforce.py</code>	33 tests, no API key required